

Practical 1 – End-to-End Supervised Learning using ANN

Architecture:

Input → Dense(64, ReLU) → Dense(32, ReLU) → Output

Dataset: Breast Cancer dataset (binary classification)

```
# Cell 1: Import libraries
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.metrics import f1_score, confusion_matrix, ConfusionMatrixDisplay
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
# Cell 2: Load dataset
```

```
data = load_breast_cancer()
```

```
X = data.data
y = data.target
```

```
print("X shape:", X.shape)
print("y shape:", y.shape)
# Cell 3: Train-test split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
# Cell 4: Data preprocessing
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# Cell 5: Build ANN model
```

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])
```

```

model.summary()
# Cell 6: Compile model

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
# Cell 7: Train model

history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
# Cell 8: Prediction

y_prob = model.predict(X_test)

y_pred = (y_prob > 0.5).astype(int).flatten()
# Cell 9: Evaluation

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
# Cell 10: Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=data.target_names
)

disp.plot()
plt.show()

```

Extra Task – Change Hidden Layer Neurons

Original:

64 → 32

Change it to:

128 → 64

Only change the model:

```
model2 = Sequential([
    Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(64, activation='relu'),
    Dense(1, activation='sigmoid')
])

model2.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model2.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2
)
```

Then compare the **test accuracy** of Model 1 and Model 2.

This directly covers the Practical 1 requirements: preprocessing, train-test split, 64 → 32 ANN, Adam 0.001, batch size 32, 10 epochs, and accuracy/precision/recall/F1/confusion matrix.

Practical 2 – Mathematical Model of Neural Network & Forward Propagation

The practical asks for a simple network:

Input(4) → Dense(8, ReLU) → Dense(3, Softmax)

Optimizer: SGD | LR: 0.01 | Epochs: 50

Step 1 – Import libraries

```
import numpy as np
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
```

Step 2 – Create simple dataset

```
X = np.array([
    [1, 2, 3, 4],
    [2, 3, 4, 5],
    [3, 4, 5, 6],
    [4, 5, 6, 7],
    [5, 6, 7, 8],
    [6, 7, 8, 9]
])
```

```
y = np.array([
    [1, 0, 0],
    [1, 0, 0],
    [0, 1, 0],
    [0, 1, 0],
    [0, 0, 1],
    [0, 0, 1]
])
```

Step 3 – Build the neural network

```
model = Sequential([
    Dense(8, activation='relu', input_shape=(4,)),
    Dense(3, activation='softmax')
])
```

```
model.summary()
```

Step 4 – Compile the model

```
model.compile(
    optimizer=SGD(learning_rate=0.01),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Step 5 – Train the model

```
history = model.fit(
    X,
    y,
    epochs=50,
```

```
verbose=1
)
```

Step 6 – Predict output

```
prediction = model.predict(X)

print("Predicted probabilities:")
print(prediction)

print("Predicted classes:")
print(np.argmax(prediction, axis=1))
```

Manual Forward Propagation

For one neuron, the basic calculation is:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

Then apply ReLU:

$$\text{ReLU}(z) = \max(0, z)$$

Simple NumPy verification

```
# Input values
x = np.array([1, 2, 3, 4])

# Example weights and bias
w = np.array([0.1, 0.2, 0.3, 0.4])
b = 0.5

# Weighted sum
z = np.dot(x, w) + b

# ReLU
output = max(0, z)

print("Weighted Sum:", z)
print("ReLU Output:", output)
```

Expected calculation

$$z = (1)(0.1) + (2)(0.2) + (3)(0.3) + (4)(0.4) + 0.5 = (1)(0.1) + (2)(0.2) + (3)(0.3) + (4)(0.4) + 0.5$$
$$z = 0.1 + 0.4 + 0.9 + 1.6 + 0.5 = 0.1 + 0.4 + 0.9 + 1.6 + 0.5 \quad z = 3.5 \quad z = 3.5$$

Therefore,

$$\text{ReLU}(3.5) = 3.5$$

Output = 3.5

Extra Task – Replace ReLU with Sigmoid

Just change:

```
Dense(8, activation='relu')
```

to:

```
Dense(8, activation='sigmoid')
```

Complete modified model:

```
model_sigmoid = Sequential([
    Dense(8, activation='sigmoid', input_shape=(4,)),
    Dense(3, activation='softmax')
])
```

```
model_sigmoid.compile(
    optimizer=SGD(learning_rate=0.01),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
model_sigmoid.fit(
    X,
    Y,
    epochs=50,
    verbose=1
)
```

Comparison: Compare the final accuracy of the **ReLU model** and **Sigmoid model**.

Practical 3 – Effect of Neural Network Hyperparameters

This one can be kept **very simple**: use MNIST, build the given network, then run the 3 required experiments—learning rate, batch size, and hidden neurons.

Step 1 – Import libraries

```
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.optimizers import Adam
```

Step 2 – Load MNIST dataset

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)
```

Step 3 – Normalize the data

```
x_train = x_train / 255.0
x_test = x_test / 255.0
```

Step 4 – Create the baseline model

Required architecture:

Flatten → Dense(128) → Dense(64) → Dense(10)

```
def create_model(hidden1=128, hidden2=64, learning_rate=0.001):

    model = Sequential([
        Flatten(input_shape=(28, 28)),
        Dense(hidden1, activation='relu'),
        Dense(hidden2, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(
        optimizer=Adam(learning_rate=learning_rate),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

Step 5 – Baseline experiment

```
model = create_model(
    hidden1=128,
    hidden2=64,
    learning_rate=0.001
)

history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.2
)
```

Step 6 – Test accuracy

```
loss, accuracy = model.evaluate(x_test, y_test)

print("Test Accuracy:", accuracy)
```

Experiment 1 – Learning Rate = 0.01

```
model_lr = create_model(
    hidden1=128,
    hidden2=64,
    learning_rate=0.01
)

history_lr = model_lr.fit(
    x_train,
    y_train,
    epochs=10,
```

```

        batch_size=64,
        validation_split=0.2
    )
    loss, accuracy = model_lr.evaluate(x_test, y_test)

    print("Accuracy with LR = 0.01:", accuracy)

```

Experiment 2 – Batch Size = 128

```

model_batch = create_model(
    hidden1=128,
    hidden2=64,
    learning_rate=0.001
)

history_batch = model_batch.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=128,
    validation_split=0.2
)

loss, accuracy = model_batch.evaluate(x_test, y_test)

print("Accuracy with Batch Size = 128:", accuracy)

```

Experiment 3 – Hidden Neurons = 256

Here we change the first hidden layer from **128** to **256**.

```

model_neurons = create_model(
    hidden1=256,
    hidden2=64,
    learning_rate=0.001
)

history_neurons = model_neurons.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.2
)

loss, accuracy = model_neurons.evaluate(x_test, y_test)

print("Accuracy with 256 neurons:", accuracy)

```

Learning Curves

For the practical, students can plot the baseline learning curve:

```

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Learning Curve")

```



```
plt.legend()  
plt.show()
```

For loss:

```
plt.plot(history.history['loss'], label='Training Loss')  
plt.plot(history.history['val_loss'], label='Validation Loss')
```

```
plt.xlabel("Epoch")  
plt.ylabel("Loss")  
plt.title("Loss Curve")  
plt.legend()  
plt.show()
```

Simple comparison table

Students can record their results like this:

Experiment	Change	Test Accuracy
Baseline	LR = 0.001, Batch = 64, Neurons = 128	_____
Experiment 1	LR = 0.01	_____
Experiment 2	Batch Size = 128	_____
Experiment 3	Neurons = 256	_____

Conclusion:

Compare the accuracy and learning curves and state how changing the **learning rate, batch size, and hidden neurons** affected the model's performance.

Practical 4: ANN for Regression

1. Import Libraries

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
```

2. Load Dataset

Example using the California Housing dataset:

```
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()

X = data.data
y = data.target

print(X.shape)
print(y.shape)
```

3. Split the Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

4. Standardize the Features

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

5. Build ANN Model

Architecture:

Input → Dense(64, ReLU) → Dense(32, ReLU) → Dense(1, Linear)

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1, activation='linear')
])
```

6. Compile the Model

```
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)
```

7. Train the Model

```
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.2
)
```

8. Make Predictions

```
y_pred = model.predict(X_test)
y_pred = y_pred.flatten()
```

9. Evaluate the Model

```
mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))

r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

10. Student Extra Task: One Hidden Layer

Remove the 32-neuron hidden layer:

```
model_simple = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(1, activation='linear')
])

model_simple.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

history_simple = model_simple.fit(
    X_train,
```

```

y_train,
epochs=10,
batch_size=64,
validation_split=0.2
)

```

Evaluate:

```

y_pred_simple = model_simple.predict(X_test).flatten()

mae_simple = mean_absolute_error(y_test, y_pred_simple)
rmse_simple = np.sqrt(mean_squared_error(y_test, y_pred_simple))
r2_simple = r2_score(y_test, y_pred_simple)

print("One Hidden Layer")
print("MAE:", mae_simple)
print("RMSE:", rmse_simple)
print("R2 Score:", r2_simple)

```

Comparison

```

print("Original Model")
print("MAE:", mae)
print("RMSE:", rmse)
print("R2:", r2)

print("\nOne Hidden Layer Model")
print("MAE:", mae_simple)
print("RMSE:", rmse_simple)
print("R2:", r2_simple)

```

Conclusion: Compare the two models using MAE, RMSE and R². Lower MAE/RMSE and higher R² indicate better regression performance.

Practical 5: CNN Architecture Design for CIFAR-10 — Answer Key

1. Import Libraries

```

import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.optimizers import Adam

```

2. Load CIFAR-10 Dataset

```

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

```

```
print(x_train.shape)
print(x_test.shape)
```

3. Normalize Images

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

4. Build Baseline CNN

Architecture:

Conv(32) → Conv(32) → MaxPool → Conv(64) → Conv(64) → MaxPool → Flatten → Dense → Softmax

```
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    Conv2D(32, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),

    Conv2D(64, (3, 3), activation='relu'),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
```

5. Compile the Model

```
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

6. Train the Model

```
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
```

7. Evaluate the Model

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)

print("Test Accuracy:", test_accuracy)
```

8. Check Parameter Count

```
model.summary()
```

The summary displays the **number of parameters** in the CNN.

Student Extra Task: Add One Convolutional Layer

Add another convolutional layer, for example Conv2D(64) before the second MaxPooling layer.

```
model_extra = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    Conv2D(32, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),

    Conv2D(64, (3, 3), activation='relu'),
    Conv2D(64, (3, 3), activation='relu'),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
```

Compile

```
model_extra.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Train

```
history_extra = model_extra.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
```

Evaluate

```
test_loss_extra, test_accuracy_extra = model_extra.evaluate(
    x_test, y_test
)
```

```
print("Extra Layer Test Accuracy:", test_accuracy_extra)
```

Compare Parameter Count

```
print("Baseline Model Parameters:")
print(model.count_params())
```

```
print("Extra Layer Model Parameters:")
print(model_extra.count_params())
```

Compare Learning Curves

```
plt.plot(history.history['accuracy'], label='Baseline Training')
plt.plot(history.history['val_accuracy'], label='Baseline Validation')
plt.plot(history_extra.history['accuracy'], label='Extra Layer Training')
plt.plot(history_extra.history['val_accuracy'], label='Extra Layer Validation')

plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Learning Curve Comparison')
```

```
plt.legend()  
plt.show()
```

Final Comparison

Students should compare:

Measure	Baseline CNN	CNN with Extra Conv Layer
Test Accuracy	Record value	Record value
Number of Parameters	Record value	Record value
Training Accuracy	Record from graph	Record from graph
Validation Accuracy	Record from graph	Record from graph

Conclusion: The student should observe how adding one convolutional layer changes the **model complexity, number of parameters, accuracy, and learning curves**.

Practical 6: Transfer Learning and Fine-Tuning using Pretrained CNN

1. Import Libraries

```
import tensorflow as tf

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GlobalAveragePooling2D, Dropout, Dense
from tensorflow.keras.optimizers import Adam
```

2. Load CIFAR-10 Dataset

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

3. Resize Images

MobileNetV2 normally uses larger input images, so resize CIFAR-10 images.

```
x_train = tf.image.resize(x_train, (96, 96))
x_test = tf.image.resize(x_test, (96, 96))
```

4. Load Pretrained MobileNetV2

```
base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(96, 96, 3)
)
```

```
base_model.trainable = False
```

5. Build the Model

```
model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dropout(0.2),
    Dense(10, activation='softmax')
])
```

6. Compile the Model

```
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

7. Train the Classification Head


```
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
```

8. Evaluate

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)
```

```
print("Test Accuracy:", test_accuracy)
```

9. Check Trainable and Non-Trainable Parameters

```
model.summary()
```

Fine-Tuning

Unfreeze selected deeper layers of MobileNetV2.

```
base_model.trainable = True
```

```
for layer in base_model.layers[:-20]:
    layer.trainable = False
```

Now only the **last 20 layers** of the pretrained network are trainable.

Compile with Lower Learning Rate

```
model.compile(
    optimizer=Adam(learning_rate=0.00001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Fine-Tune

```
history_fine = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
```

Evaluate Again

```
fine_loss, fine_accuracy = model.evaluate(x_test, y_test)
```

```
print("Fine-Tuned Accuracy:", fine_accuracy)
```

Student Extra Task

```
print("Total Parameters:", model.count_params())
print("Trainable Parameters:",
      sum(tf.keras.backend.count_params(w) for w in model.trainable_weights))
print("Non-Trainable Parameters:",
      sum(tf.keras.backend.count_params(w) for w in
model.non_trainable_weights))
```

Compare:

Measure	Frozen Model	Fine-Tuned Model
Test Accuracy	Record	Record
Trainable Parameters	Record	Record
Non-Trainable Parameters	Record	Record

Conclusion: Fine-tuning makes selected pretrained layers trainable and allows the model to adapt its learned features to the new dataset.

Practical 7: Siamese Neural Network for Image Similarity

For simplicity, use **MNIST** because creating image pairs is easy.

1. Import Libraries

```
import numpy as np
import tensorflow as tf

from tensorflow.keras.models import Model
from tensorflow.keras.layers import (
    Input, Conv2D, MaxPooling2D,
    GlobalAveragePooling2D, Dense,
    Lambda
)
```

2. Load MNIST

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)

print(x_train.shape)
```

3. Create Positive and Negative Pairs

```
def create_pairs(images, labels, num_pairs=10000):

    pairs = []
    pair_labels = []

    for i in range(num_pairs):

        # Positive pair
        idx1 = np.random.randint(0, len(images))
        same_class = np.where(labels == labels[idx1])[0]
        idx2 = np.random.choice(same_class)

        pairs.append([images[idx1], images[idx2]])
        pair_labels.append(1)

        # Negative pair
        idx1 = np.random.randint(0, len(images))
        different_class = np.where(labels != labels[idx1])[0]
        idx2 = np.random.choice(different_class)
```

```

pairs.append([images[idx1], images[idx2]])
pair_labels.append(0)

```

```

return np.array(pairs), np.array(pair_labels)

```

4. Create Training Pairs

```

pairs, pair_labels = create_pairs(
    x_train,
    y_train,
    5000
)

```

```

print(pairs.shape)
print(pair_labels.shape)

```

Here:

- 1 = similar/same class
- 0 = different class

5. Create Shared CNN Branch

```

input_shape = (28, 28, 1)

```

```

input_img = Input(shape=input_shape)

```

```

x = Conv2D(32, (3, 3), activation='relu')(input_img)
x = MaxPooling2D((2, 2))(x)

```

```

x = Conv2D(64, (3, 3), activation='relu')(x)

```

```

x = GlobalAveragePooling2D()(x)

```

```

embedding = Dense(64, activation='relu')(x)

```

```

embedding_model = Model(input_img, embedding)

```

6. Create Two Shared Branches

```

input1 = Input(shape=input_shape)
input2 = Input(shape=input_shape)

```

```

embedding1 = embedding_model(input1)
embedding2 = embedding_model(input2)

```

Both inputs use the **same CNN**, therefore the weights are shared.

7. Calculate Distance

```

distance = Lambda(
    lambda x: tf.abs(x[0] - x[1])
)([embedding1, embedding2])

```

8. Binary Output

```

output = Dense(1, activation='sigmoid')(distance)

```

```

siamese_model = Model(
    inputs=[input1, input2],
    outputs=output
)

```

```
)
```

9. Compile

```
siamese_model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

10. Train

```
history = siamese_model.fit(  
    [pairs[:, 0], pairs[:, 1]],  
    pair_labels,  
    epochs=5,  
    batch_size=64,  
    validation_split=0.2  
)
```

11. Test Random Pairs

```
test_pairs, test_labels = create_pairs(  
    x_test,  
    y_test,  
    1000  
)  
  
loss, accuracy = siamese_model.evaluate(  
    [test_pairs[:, 0], test_pairs[:, 1]],  
    test_labels  
)
```

```
print("Test Accuracy:", accuracy)
```

Student Extra Task: Change Embedding Size to 32

Change:

```
embedding = Dense(64, activation='relu')(x)  
to:
```

```
embedding = Dense(32, activation='relu')(x)
```

Train the model again and compare:

Measure	Embedding = 64	Embedding = 32
Test Accuracy	Record	Record
Parameters	Record	Record
Training Loss	Record	Record

Conclusion: The Siamese network learns an embedding representation and compares two images using their embedding distance.

Practical 8: CNN Numerical Analysis and Architectural Modification

No code for this just the numerical to be solved. Values will be given at the time of examination.

Practical 9: Kalman Filter for Noisy Sensor Data

1. Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
```

2. Generate True and Noisy Sensor Data

```
np.random.seed(42)

n = 100

true_signal = np.linspace(0, 10, n)

noise = np.random.normal(0, 1.5, n)

measurements = true_signal + noise

plt.figure(figsize=(10, 5))
plt.plot(true_signal, label="True Signal")
plt.scatter(range(n), measurements, label="Noisy Measurements", s=15)

plt.xlabel("Time")
plt.ylabel("Value")
plt.title("True and Noisy Sensor Data")
plt.legend()
plt.show()
```

3. Initialize Kalman Filter

Given:

- Initial covariance $P = 1$
- Process noise $Q = 0.01$
- Measurement noise $R = 1.5^2 = 2.25$

```
P = 1.0
Q = 0.01
R = 1.5 ** 2
```

```
x_est = 0.0
```

4. Prediction and Measurement Update

The Kalman filter has two main stages:

Prediction → Measurement Update

```
estimates = []

for z in measurements:

    # Prediction
    x_pred = x_est
    P_pred = P + Q

    # Measurement Update
    K = P_pred / (P_pred + R)

    x_est = x_pred + K * (z - x_pred)

    P = (1 - K) * P_pred

    estimates.append(x_est)
```

Here:

- z = noisy measurement
- K = Kalman Gain
- x_{est} = estimated value
- P = estimation covariance

5. Plot the Results

```
plt.figure(figsize=(10, 5))

plt.plot(true_signal, label="True Signal")
plt.scatter(
    range(n),
    measurements,
    label="Noisy Signal",
    s=15
)
plt.plot(
    estimates,
    label="Kalman Estimate",
    linewidth=2
)

plt.xlabel("Time")
plt.ylabel("Value")
plt.title("Kalman Filter Result")
plt.legend()
plt.show()
```

6. Calculate RMSE

```
rmse_noisy = np.sqrt(
    np.mean((true_signal - measurements) ** 2)
```

```
)

rmse_filtered = np.sqrt(
    np.mean((true_signal - estimates) ** 2)
)

print("RMSE Before Filtering:", rmse_noisy)
print("RMSE After Filtering:", rmse_filtered)

```

The filtered RMSE should generally be lower than the noisy-signal RMSE.

Student Extra Task

Change Q and R and observe the effect.

Case 1: Original

```
Q = 0.01
R = 2.25

```

Case 2: Higher Q

```
Q = 1.0
R = 2.25

```

Case 3: Lower R

```
Q = 0.01
R = 0.25

```

Interpretation

- **Higher Q** → filter assumes the system can change more → estimate responds more quickly to measurements.
- **Higher R** → filter assumes measurements are noisier → trusts measurements less.
- **Lower R** → filter assumes measurements are more reliable → trusts measurements more.

Therefore, **lower R makes the Kalman filter trust measurements more.**

Practical 10: Vanilla RNN for Time-Series Forecasting

For a simple demonstration, we can create a synthetic monthly time series.

1. Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.metrics import mean_absolute_error, mean_squared_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
from tensorflow.keras.optimizers import Adam

```

2. Create Time Series

```
np.random.seed(42)

```

```

time = np.arange(120)

data = (
    np.sin(2 * np.pi * time / 12)
    + 0.05 * np.random.randn(120)
)

plt.figure(figsize=(10, 5))
plt.plot(data)

plt.xlabel("Time")
plt.ylabel("Value")
plt.title("Time Series Data")
plt.show()

```

3. Create Sliding-Window Sequences

Window size = 12

```

window_size = 12

X = []
y = []

for i in range(len(data) - window_size):
    X.append(data[i:i + window_size])
    y.append(data[i + window_size])

X = np.array(X)
y = np.array(y)

print("X shape:", X.shape)
print("y shape:", y.shape)

```

4. Reshape Input for RNN

RNN input format is:

Samples × Time Steps × Features

```

X = X.reshape((X.shape[0], X.shape[1], 1))

print(X.shape)

```

5. Train-Test Split

```

split = int(0.8 * len(X))

X_train = X[:split]
X_test = X[split:]

y_train = y[:split]
y_test = y[split:]

```

6. Build Vanilla RNN

Architecture:

Sequence → SimpleRNN(32, tanh) → Dense(1, Linear)


```

model = Sequential([
    SimpleRNN(
        32,
        activation='tanh',
        input_shape=(window_size, 1)
    ),
    Dense(1, activation='linear')
])

```

7. Compile the Model

```

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

```

8. Train the Model

```

history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2
)

```

9. Make Predictions

```

y_pred = model.predict(X_test)

y_pred = y_pred.flatten()

```

10. Calculate MAE and RMSE

```

mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(
    mean_squared_error(y_test, y_pred)
)

print("MAE:", mae)
print("RMSE:", rmse)

```

11. Plot Actual vs Predicted

```

plt.figure(figsize=(10, 5))

plt.plot(y_test, label="Actual")
plt.plot(y_pred, label="Predicted")

plt.xlabel("Time")
plt.ylabel("Value")
plt.title("Actual vs Predicted Time Series")
plt.legend()
plt.show()

```

Student Extra Task: Window Size 24

Change:

```
window_size = 12
```

to:

```
window_size = 24
```

Then recreate the sequences:

```
X = []
```

```
y = []
```

```
for i in range(len(data) - window_size):  
    X.append(data[i:i + window_size])  
    y.append(data[i + window_size])
```

```
X = np.array(X)
```

```
y = np.array(y)
```

```
X = X.reshape((X.shape[0], X.shape[1], 1))
```

Split again:

```
split = int(0.8 * len(X))
```

```
X_train = X[:split]
```

```
X_test = X[split:]
```

```
y_train = y[:split]
```

```
y_test = y[split:]
```

Build the same RNN:

```
model_24 = Sequential([  
    SimpleRNN(  
        32,  
        activation='tanh',  
        input_shape=(24, 1)  
    ),  
    Dense(1, activation='linear')  
])
```

Compile and train:

```
model_24.compile(  
    optimizer=Adam(learning_rate=0.001),  
    loss='mse',  
    metrics=['mae']  
)
```

```
history_24 = model_24.fit(  
    X_train,  
    y_train,  
    epochs=10,  
    batch_size=32,  
    validation_split=0.2  
)
```

Evaluate:

```

y_pred_24 = model_24.predict(X_test).flatten()

mae_24 = mean_absolute_error(y_test, y_pred_24)

rmse_24 = np.sqrt(
    mean_squared_error(y_test, y_pred_24)
)

print("Window Size 24")
print("MAE:", mae_24)
print("RMSE:", rmse_24)

```

Comparison

Measure	Window = 12	Window = 24
MAE	Record value	Record value
RMSE	Record value	Record value
Training Samples	Record	Record
Forecasting Performance	Compare	Compare

Conclusion

The student should compare **MAE and RMSE** for window sizes 12 and 24. A lower MAE/RMSE indicates better forecasting performance. The actual and predicted values should also be compared using the graph.